

TASK- 4 A

(one Esp act as transmitter and the other acts as receiver)

Task: esp to esp connection that is an ultrasonic sensor(esp1) is connected to one esp and the other esp is connected to a relay module so when the ultrasonic sensor measures a distance within a range the relay connected to the other esp should be turned on and also the command for turning on or off of relay should be shown by the esp1 and the

Step 1: Get the Receiver's MAC Address

First, boot up **Receiver ESP32** in Thonny and run this quick script in the Shell to find its hardware address.

```
import network

# Activate wireless interface
w0 = network.WLAN(network.STA_IF)
w0.active(True)

# Print MAC Address in standard hex format
mac = w0.config('mac')
print("Receiver MAC Address:", ':'.join(['%02x' % b for b in mac]))
```

Note down the printed address (e.g., 24:0a:c4:xx:xx:xx). You will need to paste this string into the transmitter script.

Step 2: The Transmitter Code (Ultrasonic Sensor)

Connect Ultrasonic sensor to the Transmitter ESP32:

- **Trig** → GPIO 5
- **Echo** → GPIO 18

Create a new file in Thonny, copy this code, update your receiver's MAC address, and save it as `main.py` on the Transmitter ESP32.

```
import machine
import time
import network
import espnow

# 1. Configure Pins for Ultrasonic Sensor
trig = machine.Pin(5, machine.Pin.OUT)
```

```

echo = machine.Pin(18, machine.Pin.IN)

# 2. Setup ESP-NOW Wireless Communication
w0 = network.WLAN(network.STA_IF)
w0.active(True)

e = espnow.ESPNow()
e.active(True)

# REPLACE WITH YOUR RECEIVER'S MAC ADDRESS (Keep the b'' format)
peer_mac = b'\x24\x0a\xc4\x00\x00\x00'
e.add_peer(peer_mac)

def get_distance():
    # Ensure trig pin is low
    trig.value(0)
    time.sleep_us(2)

    # Send a 10-microsecond pulse to trigger the sensor
    trig.value(1)
    time.sleep_us(10)
    trig.value(0)

    # Measure the duration of the incoming high pulse on echo pin
    # timeout=30000us covers roughly up to 5 meters
    duration = machine.time_pulse_us(echo, 1, 30000)

    if duration < 0:
        return 999.0 # Return an out-of-range value if pulse timeout
occurs

    # Calculate distance in cm (Speed of sound is ~0.0343 cm/us)
    distance = (duration * 0.0343) / 2
    return distance

print("Transmitter is ready...")

while True:
    dist = get_distance()
    print("Measured Distance: {:.2f} cm".format(dist))

    # Check if the target is within the 20 to 30 cm range
    if dist <= 30.0:
        message = "ON"
    else:
        message = "OFF"

    # Send the message string to the receiver board

```

```

try:
    e.send(peer_mac, message)
    print(f"Sent command: {message}")
except Exception as err:
    print("Error sending data:", err)

time.sleep(0.5) # Sample every 500ms

```

Step 3: The Receiver Code (Relay Module)

Connect your Relay Module to the Receiver ESP32:

- **IN / Signal Pin**  GPIO 25

Save this code as main.py on your Receiver ESP32 using Thonny.

```

import machine
import network
import espnow

# 1. Configure Pin for Relay
relay = machine.Pin(25, machine.Pin.OUT)
relay.value(0) # Start with the relay turned OFF

# 2. Setup ESP-NOW Wireless Communication
w0 = network.WLAN(network.STA_IF)
w0.active(True)

e = espnow.ESPNow()
e.active(True)

print("Receiver is waiting for data...")

while True:
    # Check if any messages have arrived (blocking read)
    host, msg = e.recv()

    if msg:
        # Decode bytes message to string
        command = msg.decode('utf-8')
        print(f"Received from {host}: {command}")

        # Action based on command received
        if command == "ON":
            relay.value(1) # Turn Relay ON
            print("-> Relay is ON")
        elif command == "OFF":
            relay.value(0) # Turn Relay OFF
            print("-> Relay is OFF")

```

NOTE :

1. **Formatting the MAC address:** Notice how in the transmitter code the MAC address colon separators are replaced with backslashes (\x). If your address is 24:0a:c4:11:22:33, write it exactly as b'\x24\x0a\xc4\x11\x22\x33'.

Give this a run in your workshop and see how it handles!